

Reescritura de consultas

Clase 16

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Clustered versus unclustered (anterior)

Parser y preprocesamiento

Algebra relacional extendido

Reescritura de una consulta

Reglas de reescritura

Outline

Clustered versus unclustered (anterior)

Parser y preprocesamiento

Algebra relacional extendido

Reescritura de una consulta

Reglas de reescritura

Clustered versus unclustered: componentes de un índice

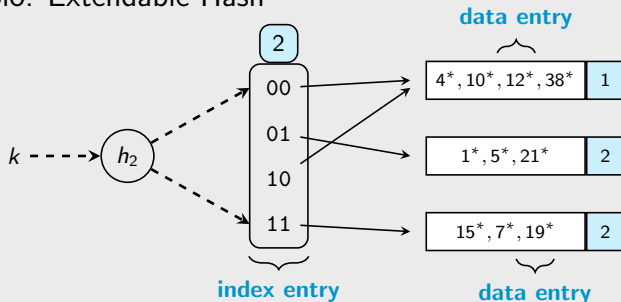
Definición

Search key := parámetro de **búsqueda**

Index entry := valor o puntero de la **estructura** de un índice

Data entry := **tupla** o **dirección** donde se almacena una tupla

Ejemplo: Extendable Hash



Clustered versus unclustered: definición

Definición

Search key := parámetro de **búsqueda**

Index entry := valor o puntero de la **estructura** de un índice

Data entry := **tupla** o **dirección** donde se almacena una tupla

Definición

Clustered index := índice para el cual el orden de sus data entries es el **mismo orden de las tuplas** en disco.

Unclustered index := índice que **NO** es clustered.

En clustered index: **data entries = tuplas**

Clustered versus unclustered: ejemplos

Definición

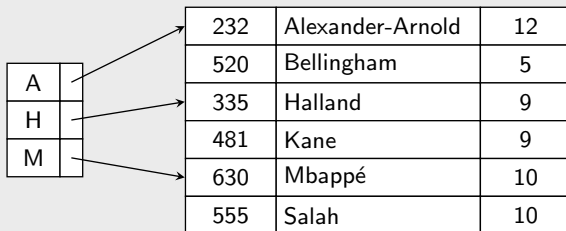
Clustered index := índice para el cual el orden de sus data entries es el **mismo orden de las tuplas** en disco.

Unclustered index := índice que **NO** es clustered.

¿clustered o unclustered?

Players(ID, name, number)

A		232	Alexander-Arnold	12
H		520	Bellingham	5
M		335	Halland	9
		481	Kane	9
		630	Mbappé	10
		555	Salah	10



Clustered versus unclustered: ejemplos

Definición

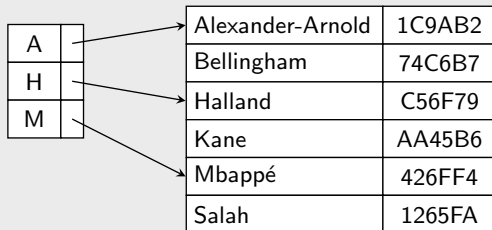
Clustered index := índice para el cual el orden de sus data entries es el **mismo orden de las tuplas** en disco.

Unclustered index := índice que **NO** es clustered.

¿clustered o unclustered?

Players(ID, name, number)

A		Alexander-Arnold	1C9AB2
H		Bellingham	74C6B7
M		Halland	C56F79
		Kane	AA45B6
		Mbappé	426FF4
		Salah	1265FA



Clustered versus unclustered: comentarios

- En general, por cada relación es posible mantener **un solo índice clustered**. (¿por que?)
- Índice unclustered son **ineficientes** cuando el output es numeroso.
 - Ejemplo: consultas por rango
- Usualmente, clustered y unclustered son conocidos como:
 - clustered index = índice **primario**.
 - unclustered index = índice **secundario**.

Usualmente, índice **clustered** esta reservado para la PRIMARY KEY

(depende del motor de BD)

Outline

Clustered versus unclustered (anterior)

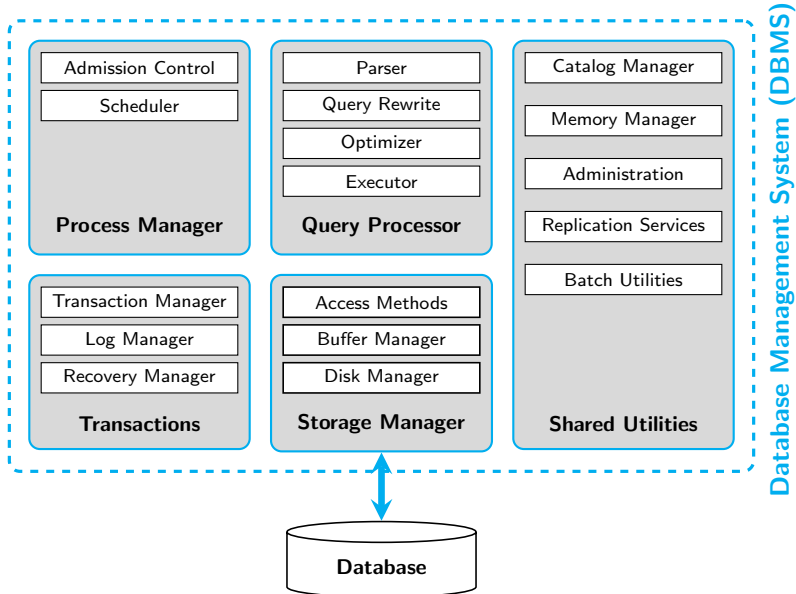
Parser y preprocesamiento

Algebra relacional extendido

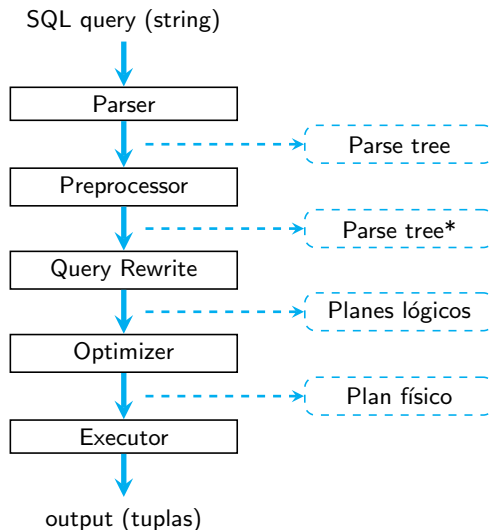
Reescritura de una consulta

Reglas de reescritura

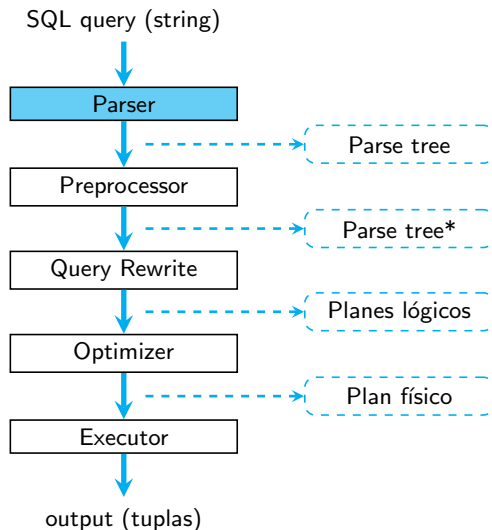
Procesamiento de consultas



Zoom al procesador de consultas



Zoom al procesador de consultas



Sintaxis y semántica de un lenguaje

Definición

1. La **sintaxis** de una lenguaje es un conjunto de reglas que definen los strings válidos que tienen significado.

¿son estas consultas válidas en SQL?

- `SELECT FROM Players T1 AND T2 WHERE age = 30`
- `SELECT * FROM Players, Matches WHERE age = 30`

Sintaxis y semántica de un lenguaje

Definición

1. La **sintaxis** de un lenguaje es un conjunto de reglas que definen los strings válidos que tienen significado.
2. La **semántica** de un lenguaje define el significado de un string correcto según la sintaxis.

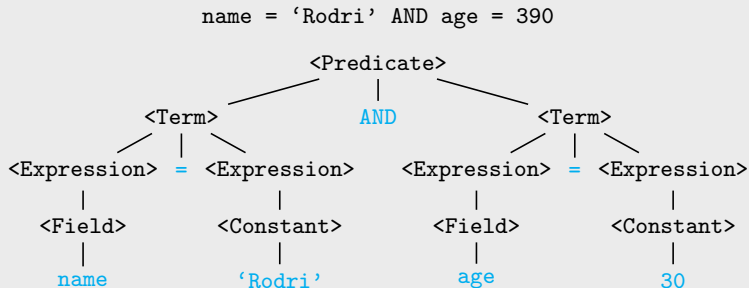
¿cuál es la semántica de esta consulta?

```
SELECT * FROM Players, Matches WHERE age = 30
```

Parser

1. Encargado de **verificar la sintaxis** de SQL de una consulta.
2. Entregar la estructura de una consulta: **árbol de parsing**

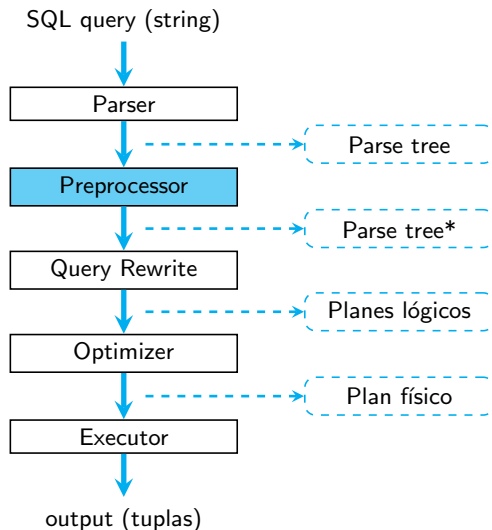
Ejemplo de árbol de parsing



Para más detalles ver curso:

Teoría de autómatas y lenguajes formales

Zoom al procesador de consultas



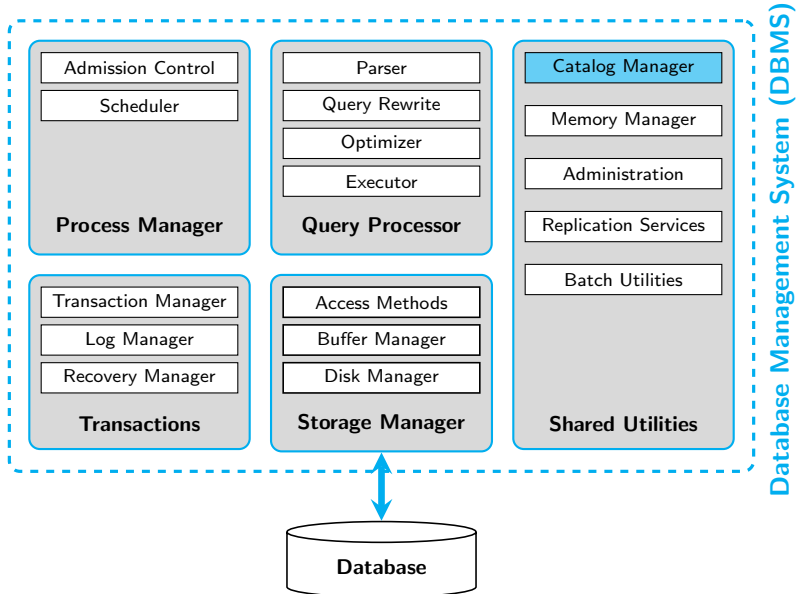
Preprocesamiento de consultas

Para cada consulta

- Verificar los nombres de **tablas**
- Verificar los nombres de **atributos**
- Verificar el uso correcto de **tipos**
- Calcular **expresiones** simples
- ...

Para verificar esta información: **catalogo del sistema**

Catálogo del sistema



Catálogo del sistema

Definición

- Los **metadatos** es la información de la BD fuera de su contenido.
 - Ejemplo: esquema de la base de datos.
- El **catálogo del sistema** almacena los metadatos de la BD.

Metadatos

1. Información sobre las **relaciones**.
2. **Estadísticas** del contenido.
3. Información sobre los **índices**.

Catálogo del sistema

Metadatos

1. Información sobre las **relaciones**
2. **Estadísticas** del contenido
3. Información sobre los **índices**

Catálogo del sistema

Metadatos

1. Información sobre las **relaciones**

■ Relaciones

- Nombre
- Atributos
- Dirección de almacenamiento
- Restricciones de integridad

■ Atributos

- Nombre
- Tipo de dato
- Tamaño
- Detalles de almacenamiento

2. **Estadísticas** del contenido

3. Información sobre los **índices**

Catálogo del sistema

Metadatos

1. Información sobre las **relaciones**
2. **Estadísticas** del contenido
 - Número de páginas
 - Número de tuplas
 - Tamaño promedio de una tupla
 - Para cada atributo:
 - Cantidad de valores distintos
 - Tamaño promedio
 - Distribución de los datos (histogramas)
 - Valores máximos y mínimos
3. Información sobre los **índices**

Catálogo del sistema

Metadatos

1. Información sobre las **relaciones**
2. **Estadísticas** del contenido
3. Información sobre los **índices**
 - Nombre del índice
 - Atributo indexado
 - Tipo de índice
 - Tamaño del índice
 - Altura (B+tree)
 - Largo de listas (Hashing)
 - Costo promedio del índice

¿dónde se almacena el catálogo?

Metadatos son almacenados en **relaciones**

Ejemplo para listar todas las tablas

- SQLite

```
SELECT name  
FROM sqlite_schema;
```

- PostgreSQL

```
SELECT tablename  
FROM pg_catalog.pg_tables;
```

Outline

Clustered versus unclustered (anterior)

Parser y preprocesamiento

Algebra relacional extendido

Reescritura de una consulta

Reglas de reescritura

Operaciones del álgebra relacional

1. Unión $\cup$
2. Intersección $\cap$
3. Diferencia $\setminus$
4. Selección σ_θ
5. Renombrar $\rho_{A \rightarrow B}$
6. Proyección $\pi_{B_1, \dots, B_k}$
7. Join $\bowtie$
8. Producto cruz $\times$

Operaciones del álgebra relacional **extendida**

1. Unión $\cup$
2. Intersección $\cap$
3. Diferencia $\setminus$
4. Selección σ_θ
5. Renombrar $\rho_{A \rightarrow B}$
6. Proyección $\pi_{B_1, \dots, B_k}$
7. Join $\bowtie$
8. Producto cruz $\times$
9. θ -Join $\bowtie_\theta$
10. Equijoin $\bowtie_\alpha$
11. Eliminación duplicados δ
12. Agregación $\gamma_{G,L}$
13. Sorting τ

Operadores **1** al **7** en dos versiones: sin y con **duplicados**

Operaciones del álgebra relacional **extendida**

9. θ -Join $\bowtie_{\theta}$

$$\mathcal{R}_1 \bowtie_{\theta} \mathcal{R}_2 := \sigma_{\theta}(\mathcal{R}_1 \times \mathcal{R}_2)$$

Predicados θ

$$\theta \quad := \quad A \sim a \mid A \sim B \mid \theta \wedge \theta \mid \theta \vee \theta$$

- A y B son atributos cualquiera
- $\sim \in \{=, \neq, <, >, \leq, \geq, \dots\}$

Operaciones del álgebra relacional **extendida**

9. θ -Join $\bowtie_{\theta}$

$$\mathcal{R}_1 \bowtie_{\theta} \mathcal{R}_2 := \sigma_{\theta}(\mathcal{R}_1 \times \mathcal{R}_2)$$

10. Equijoin $\bowtie_{\alpha}$

$$\mathcal{R}_1 \bowtie_{\alpha} \mathcal{R}_2 := \sigma_{\alpha}(\mathcal{R}_1 \times \mathcal{R}_2)$$

Predicados de igualdad α

$$\alpha \quad := \quad A = B \mid \alpha \wedge \alpha$$

- A y B son atributos

Operaciones del álgebra relacional **extendida**

9. θ -Join $\bowtie_{\theta}$

$$\mathcal{R}_1 \bowtie_{\theta} \mathcal{R}_2 := \sigma_{\theta}(\mathcal{R}_1 \times \mathcal{R}_2)$$

10. Equijoin $\bowtie_{\alpha}$

$$\mathcal{R}_1 \bowtie_{\alpha} \mathcal{R}_2 := \sigma_{\alpha}(\mathcal{R}_1 \times \mathcal{R}_2)$$

11. Eliminación duplicados δ

$$\delta(\mathcal{R}) := \underbrace{\{\bar{t} \mid \bar{t} \in \mathcal{R}\}}_{\mathcal{R} \text{ sin duplicados}}$$

δ es equivalente a SELECT DISTINCT

Operaciones del álgebra relacional **extendida**

11. Eliminación duplicados δ

$$\delta(\mathcal{R}) := \underbrace{\{\bar{t} \mid \bar{t} \in \mathcal{R}\}}_{\mathcal{R} \text{ sin duplicados}}$$

12. Agregación $\gamma_{G,L}$

- G lista de atributos para **agrupar** (como en GROUP BY)
- L lista de atributos de **agregación** de la forma:

$$f_1(A_1), \dots, f_k(A_k)$$

donde A_i es un atributo y $f_i \in \{\text{MIN}, \text{MAX}, \text{SUM}, \dots\}$

$\gamma_{G,L}$ es **equivalente** a GROUP BY con agregación

Operaciones del álgebra relacional **extendida**

11. Eliminación duplicados δ

$$\delta(\mathcal{R}) := \underbrace{\{\bar{t} \mid \bar{t} \in \mathcal{R}\}}_{\mathcal{R} \text{ sin duplicados}}$$

12. Agregación $\gamma_{G,L}$

- G lista de atributos para **agrupar** (como en GROUP BY)
- L lista de atributos de **agregación** de la forma:

$$f_1(A_1), \dots, f_k(A_k)$$

donde A_i es un atributo y $f_i \in \{\text{MIN}, \text{MAX}, \text{SUM}, \dots\}$

13. Sorting $\tau \equiv \text{ORDER BY}$

Semántica de estos operadores **NO** es estándar

Outline

Clustered versus unclustered (anterior)

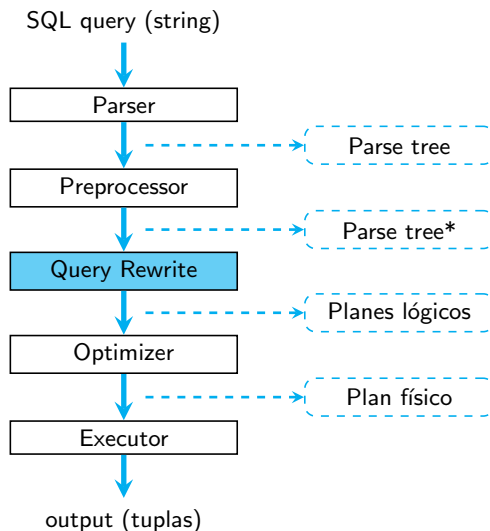
Parser y preprocesamiento

Algebra relacional extendido

Reescritura de una consulta

Reglas de reescritura

Zoom al procesador de consultas



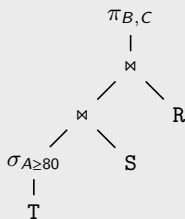
Plan lógico

Definición

Un **plan lógico** es un árbol ordenado y etiquetado tal que:

- los nodos **hojas** son relaciones y
- los nodos **internos** son operadores del álgebra relacional (extendido)

Ejemplo de plan lógico



Plan lógico define la **semántica de la consulta SQL**

(y no como computar la consulta)

¿cómo convertimos un árbol de parsing en un plan lógico?

```
SELECT  <SelList>  
FROM    <FromList>  
WHERE   <Condition>
```

1. Combinamos las relaciones en el <FromList> con **productos cruz**:

$$R_1 \times \dots \times R_n$$

2. Aplicamos **selección** con la condición dada en el <Condition>:

$$\sigma_{\langle \text{Condition} \rangle} (R_1 \times \dots \times R_n)$$

3. Aplicamos **proyección** con los atributos dados en el <SelList>:

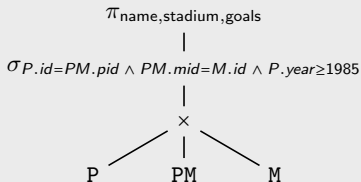
$$\pi_{\langle \text{SelList} \rangle} (\sigma_{\langle \text{Condition} \rangle} (R_1 \times \dots \times R_n))$$

¿cómo convertimos un árbol de parsing en un plan lógico?

```
SELECT    <SelList>
FROM      <FromList>
WHERE     <Condition>
```

Ejemplo de traducción

```
SELECT    name, stadium, goals
FROM      Players AS P, PlayersMatches AS PM, Matches AS M
WHERE     P.id = PM.pid AND PM.mId = M.id AND P.year >= 1985
```



¿cómo convertimos un árbol de parsing en un plan lógico?

```
SELECT    <SelList>
FROM      <FromList>
WHERE     <Condition>
```

La transformación desde un árbol a un plan lógico es más compleja:

- Considera **otros operadores**

- Renombrar
- Unión, intersección, diferencia
- DISTINCT y ORDER BY
- Agregación y GROUP BY

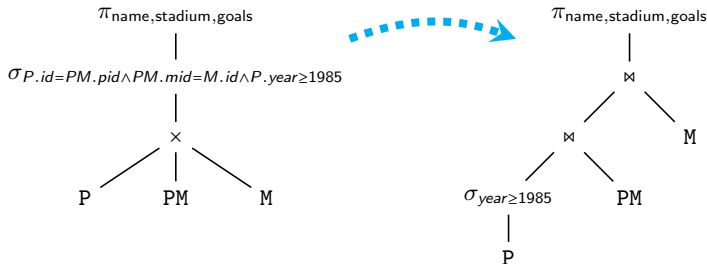
- Considera **consultas anidadas**

- EXISTS
- IN y NOT IN

En muchos casos la traducción **es casi directa** (en otros no tanto)

Reescritura de consultas

1. Convertir el árbol de parsing en un plan lógico.
2. Reescribe consulta aplicando reglas de álgebra relacional.
3. Crea un set de planes lógicos **prometedores** para ser optimizados.



Outline

Clustered versus unclustered (anterior)

Parser y preprocesamiento

Algebra relacional extendido

Reescritura de una consulta

Reglas de reescritura

Equivalencia de consultas

Definición

Las consultas Q_1 y Q_2 en álgebra relacional son **equivalentes** ($Q_1 = Q_2$) ssi:

$$Q_1(D) = Q_2(D) \quad \text{para toda base de datos } D$$

Reescritura de consultas

Dado una consulta Q , buscamos una consulta Q' tal que:

1. $Q = Q'$ y
2. evaluar Q' sea **más eficiente**.

¿cómo encontramos Q' ?

Reglas de reescritura de consultas

1. Conmutatividad
2. Asociatividad
3. Distributividad
4. Distribución de selección (push-selection)
5. Distribución de proyección (push-projection)
6. Simplificación de productos
7. Eliminación de duplicados

Una de las **mayores ventajas** del álgebra relacional

Conmutatividad

Teorema

Las siguientes consultas son **equivalentes**:

$$R \times S = S \times R$$

$$R \bowtie S = S \bowtie R$$

$$R \cap S = S \cap R$$

$$R \cup S = S \cup R$$

Estas reglas son validas tanto **con o sin duplicados**

¿cuál es la importancia de estas reglas?

Asociatividad

Teorema

Las siguientes consultas son **equivalentes**:

$$(R \times S) \times T = R \times (S \times T)$$

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

$$(R \cap S) \cap T = R \cap (S \cap T)$$

$$(R \cup S) \cup T = R \cup (S \cup T)$$

Estas reglas son validas tanto **con o sin duplicados**

¿cuál es la importancia de estas reglas?

Distributividad

Teorema

Las siguientes consultas son **equivalentes sin duplicados**:

$$R \cap (S \cup T) = (R \cap S) \cup (R \cap T)$$

¿qué ocurre **con duplicados**?

Selección: AND y OR

Teorema

Las siguientes consultas son **equivalentes**:

$$\sigma_{\theta_1 \wedge \theta_2}(R) = \sigma_{\theta_1}(\sigma_{\theta_2}(R)) = \sigma_{\theta_2}(\sigma_{\theta_1}(R))$$

$$\sigma_{\theta_1 \vee \theta_2}(R) = \underbrace{\sigma_{\theta_1}(R) \cup \sigma_{\theta_2}(R)}_{\text{sin duplicados}}$$

¿cuál es la **utilidad** de esta regla?

Selección: push-selection

Teorema

Las siguientes consultas son **equivalentes**:

$$\sigma_{\theta}(R \times S) = \sigma_{\theta}(R) \times S$$

$$\sigma_{\theta}(R \bowtie S) = \sigma_{\theta}(R) \bowtie S$$

$$\sigma_{\theta}(R \bowtie_{\theta'} S) = \sigma_{\theta}(R) \bowtie_{\theta'} S$$

$$\sigma_{\theta}(R \cap S) = \sigma_{\theta}(R) \cap S$$

si R tiene todos los atributos mencionados en θ .

Selección: push-selection

Teorema

Si R y S tiene todos los atributos mencionados en p , simultáneamente:

$$\sigma_{\theta}(R \bowtie S) = \sigma_{\theta}(R) \bowtie \sigma_{\theta}(S)$$

$$\sigma_{\theta}(R \cap S) = \sigma_{\theta}(R) \cap \sigma_{\theta}(S)$$

Las siguientes consultas son siempre **equivalentes**:

$$\sigma_{\theta}(R \cup S) = \sigma_{\theta}(R) \cup \sigma_{\theta}(S)$$

$$\sigma_{\theta}(R \setminus S) = \sigma_{\theta}(R) \setminus \sigma_{\theta}(S) = \sigma_{\theta}(R) \setminus S$$

Ejemplo: $R(A, B)$ y $S(B, C)$

$$\sigma_{B=3 \wedge B < C}(R \bowtie S)$$

Selección: push-selection

Teorema

Si R y S tiene todos los atributos mencionados en p , simultáneamente:

$$\sigma_{\theta}(R \bowtie S) = \sigma_{\theta}(R) \bowtie \sigma_{\theta}(S)$$

$$\sigma_{\theta}(R \cap S) = \sigma_{\theta}(R) \cap \sigma_{\theta}(S)$$

Las siguientes consultas son siempre **equivalentes**:

$$\sigma_{\theta}(R \cup S) = \sigma_{\theta}(R) \cup \sigma_{\theta}(S)$$

$$\sigma_{\theta}(R \setminus S) = \sigma_{\theta}(R) \setminus \sigma_{\theta}(S) = \sigma_{\theta}(R) \setminus S$$

Una de las reglas **mas importante** de los motores

Proyección: push-projection

Teorema

Las siguientes consultas son **equivalentes**:

$$\pi_L(R \bowtie S) = \pi_L(\pi_M(R) \bowtie \pi_N(S))$$

$$\pi_L(R \bowtie_{\theta} S) = \pi_L(\pi_M(R) \bowtie_{\theta} \pi_N(S))$$

$$\pi_L(R \times S) = \pi_L(\pi_M(R) \times \pi_N(S))$$

$$\pi_L(R \cup S) = \pi_L(R) \cup \pi_L(S)$$

$$\pi_L(\sigma_{\theta}(R)) = \pi_L(\sigma_{\theta}(\pi_M(R)))$$

si M y N no **interfieren** con el join ni la proyección de L .

Ejemplo: $R(a, b, c)$ y $S(b, c, d)$

$$\pi_{a,b}(R \bowtie S)$$

Joins y productos

Teorema

Las siguientes consultas son **equivalentes**:

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

$$R \bowtie S = \pi_L(\sigma_{\alpha}(R \times S))$$

donde:

$$\alpha := \bigwedge_{A \in \text{att}(R) \cap \text{att}(S)} R.A = S.A$$

$$L := \text{att}(R) \cup \text{att}(S).$$

¿cuál es la **utilidad** de esta regla?

Eliminación de duplicados

Teorema

Las siguientes consultas son **equivalentes**:

$$\delta(R \times S) = \delta(R) \times \delta(S)$$

$$\delta(R \bowtie S) = \delta(R) \bowtie \delta(S)$$

$$\delta(R \bowtie_p S) = \delta(R) \bowtie_p \delta(S)$$

$$\delta(\sigma_p(R)) = \sigma_p(\delta(R))$$

Siempre considerar que $\delta(R) = R$ si R **NO tiene duplicados** como:

- R tiene una llave primaria.
- R es el resultado de un group-by.
- R es el resultado de operadores sin duplicados.

GroupBy

- Reglas dependen mucho de las funciones de agregación.
- Algunas reglas:

$$\delta(\gamma_L(R)) = \gamma_L(R)$$

$$\gamma_L(\delta(R)) = \gamma_L(R) \quad (*)$$

(*) si L considera funciones de agregación como MAX o MIN.

¿qué reglas usar?

No existe una estrategia única

Heurísticas

- hacer **push** de las **selecciones** lo mas abajo posible
- hacer **push** de las **proyecciones** lo mas abajo posible
- proyectar y eliminar atributos que no se usarán
- convertir los productos en **equijoins**
- elegir el **mejor orden*** de joins, uniones o intersecciones
- remover eliminación de duplicados innecesarios